

OpenTrace Studio 作品介绍

项	内容
赛道	开源 AI 工具（ 自主选题 ，未绑定企业命题）
项目	OpenTrace Studio — Agent / RAG 可观测与交互调试工作台
协议	Apache-2.0
仓库	https://gitee.com/wangdandan810012/opentrace-studio
复现	pnpm install && pnpm dev → http://localhost:5173

一句话定位：把多步骤 Agent / RAG 从黑盒变成可点选、可核对证据、可回放、可本机部署的开源调试工作台——**不替代** LangSmith / Langfuse 等生产观测平台，**补齐**轻量本机 DevTools + 页内 Live Run + RAG 证据链。

一、项目背景

1.1 问题从哪里来

大模型 Agent 与 RAG 进入真实场景后，故障很少出在「模型会不会说话」，而集中在运行过程：

- **工具层：**参数错误、超时、下游 API（天气、地理编码等）失败
- **检索层：**片段分数低却仍被引用，或答案引用与证据对不上
- **规划层：**步骤过长、慢点难以定位，难以向同事或评审讲清「卡在哪一步」

过程数据往往已经存在（聊天附言、日志、平台 Trace），但开发者与教学场景仍然普遍感到：**看不清、对不上、讲不明白**。

1.2 现有做法的缺口

常见形态	能提供什么	关键不足
聊天里夹过程文本	临时可读	难筛选、难回放、难给评审结构化讲解
终端 / 文件日志	有原始数据	可视化弱，证据难与步骤绑定
云可观测控制台（如 LangSmith）	能力强、生态成熟	常绑平台与账号，开源演示与自托管成本高
开源生产平台（如 Langfuse）	可自托管、功能全	依赖较重基建，不利于「当天装上就能讲」

赛道内亦有 Trace UI 组件（如 AgentPrism）、本地 OTLP 仪表盘（如 Lookspan）、评测工具（如 Phoenix）等，说明需求真实；但缺少一个把 **Runner 真跑、四视图观测、RAG 证据质检、Diff/回放** 收成「浏览器单页可演示闭环」的开源工作台。

1.3 本项目要补的位置

面向 **开发排障、教学演示、开源评审**，OpenTrace Studio 选择：

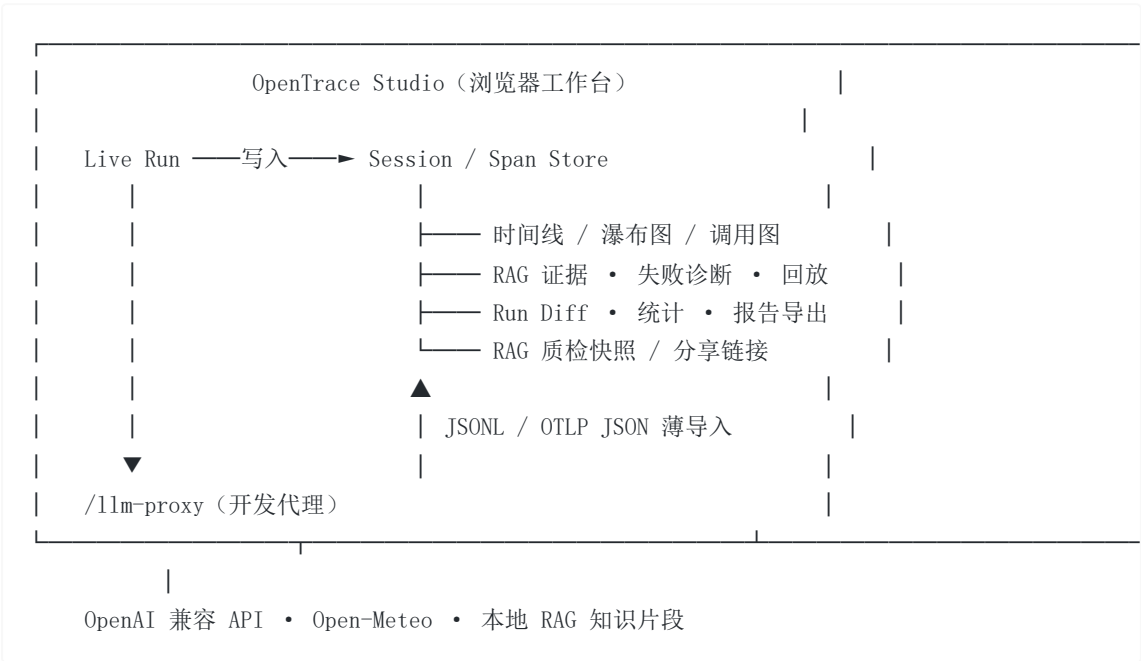
- 1. 浏览器本机运行，`pnpm dev` 即可，不强制 Docker / 账号 / 云服务
- 2. 页内 Live Run：配置 OpenAI 兼容 API 后边跑边写 Trace，流式答案与观测同步
- 3. 薄协议：JSONL 为主，OTLP JSON 子集薄导入，便于交换与二次接入
- 4. RAG 证据挂在 span 上，并提供质检快照，核对「检索—引用—回答」是否一致

选题说明：2026 上海开源软件应用创新大赛 · 开源 AI 工具赛道 · 自主选题。

二、技术架构

2.1 设计原则

薄后端、厚可视化。 观测闭环在浏览器完成；开发态仅用 Vite 插件提供 LLM 代理（规避 CORS、透传 SSE），不引入生产级采集后端。



2.2 数据模型

一次运行建模为 **Session + Span**：

实体	含义
Session	问题、答案、模型、开始时间、备注、钉选等会话级信息
Span	步骤名、kind (llm / tool / retriever / chain / custom)、起止时间、状态、入参/出参、错误、ragChunks、meta (如 tokens)

视图通过选中 span、回放进度、引用跳转联动，形成「点一处、处处更新」的调试体验。

2.3 核心模块

模块	职责
----	----

src/runner/*	Live Agent: 规划 → 地理编码 / 本地 RAG / 天气预报 → 流式作答; 边跑边写 span; 解析流式 usage
src/adapters/jsonl.ts	JSONL 导入导出
src/adapters/otlp-lite.ts	OTLP JSON 子集 → Session (含 gen_ai.usage.*、检索文档等字段映射)
src/analytics/*	会话统计与计费、关键路径、分享编解码、 RAG 质检快照
src/components/*	时间线、瀑布图、调用图、证据、回放、Diff、失败诊断、主题切换
src/storage/*	localStorage 持久化 (约 20 条) ; 脱敏辅助
vite.llm-proxy.ts	开发态代理: X-OpenTrace-Target + SSE 透传

技术栈: React 19 · Vite 6 · TypeScript · pnpm · Vitest · GitHub CI。

2.4 产品能力一览

- 1. Trace 时间线 (筛选、慢 span、LLM token 线索)
- 2. 工具调用图与瀑布图 / 关键路径
- 3. RAG 证据面板 + 答案引用跳转 + **RAG 质检快照**
- 4. 会话回放 (倍速、错误/钉选断点)
- 5. Live Run (DeepSeek / OpenAI 等预设, 流式答案与 Trace 同步)
- 6. Run Diff (耗时、状态、答案、token/粗估费用)
- 7. JSONL / OTLP 导入; Markdown / HTML / CSV 报告; 分享链接
- 8. 白天 / 夜间主题; 失败诊断分类

2.5 安全与范围边界

- API Key 仅存当前标签页 (sessionStorage), 不进仓库、不进构建产物
- 会话在 localStorage; 密钥与配置不上传本项目服务器
- Live Run 使用用户自备上游 API, 费用与条款自负
- **本届明确不做:** 万级 span 虚拟列表、完整 OTLP Collector、LangGraph / Dify 深度 SDK、账号体系与多租户云托管

三、应用场景

3.1 开发排障: 定位失败工具与慢步骤

在时间线筛选 `error` / 慢 span, 查看入参出参与右侧失败诊断 (如 Key、网络、下游 HTTP), 快速判断问题层级, 而不是在聊天记录里刷屏。

3.2 RAG 质检: 核对低分检索与误引用

点选 `retriever` span, 对照分数与原文; 从答案 `[n]` 跳回证据卡。使用顶栏 **RAG 快照** 一键汇总: 分数区间、低分片段、已引用 / 未引用 chunk, 适合贴进 Issue 或评审材料。

3.3 教学与开源评审: 本机即可演示

无需先注册云观测账号。有 Key 则真跑；无 Key 则加载离线 JSONL / OTLP 样例，配合回放讲解多步骤 Agent 过程，降低演示不确定性。

3.4 协作复现：JSONL 交换 Trace

导出 JSONL 发给同事或附在 PR/Issue；对方导入即可看到同一会话。OTLP JSON 子集也可薄导入，便于与外部埋点链路衔接（薄桥，非完整采集栈）。

3.5 实验对比：两次 Run Diff

改提示词、换模型或调整工具后，对比两次会话的耗时结构、失败点、答案摘要与粗估费用，形成可展示的迭代证据。

四、创新点

4.1 本机交互式 DevTools，而不是「再贴一段过程文本」

按前端工程师熟悉的「时间线 + 详情 + 回放」组织 Agent 过程：可筛选、可点选、可钉选、可逐步播放，解决日志可读但不可调试的问题。

4.2 RAG 证据与 span 强绑定，并给出质检快照

检索片段、分数、来源挂在对应 span；答案引用可跳转；另提供会话级 RAG 质检快照与报告段落。相对「只有 LLM 调用日志」的工具，更贴近 RAG 落地质检。

4.3 Live Run：页内真跑，边跑边写 Trace

配置兼容 API 后，规划 / 工具 / 检索 / 流式作答直接写入 Session/Span，答案与观测同步增长；流式路径解析 token usage，统计栏与 Diff 可呈现用量与粗估费用。多数方案是「先接 SDK/平台再看图」；本项目强调 **跑完就看得见**。

4.4 开源、可复现的薄协议接入

Apache-2.0；JSONL 为主协议，OTLP JSON 薄桥增强对接感，报告与分享降低演示成本。相对云产品与重型自托管平台，差异化在于：**自托管轻、RAG 证据可见、Live Run 直读 Trace**。

4.5 调试闭环：诊断 • Diff • 关键路径 • 回放

失败诊断分类、Run Diff、关键路径跳转、回放断点与会话统计，把「能跑 Demo」提升为「能解释、能对比、能复现」——贴合开源评审对**场景落地**的要求。

4.6 对标边界（避免误解）

对标对象	我们不做什么	我们强调什么
LangSmith / Langfuse	不做生产级平台与重基建	本机轻量、当天可演示
Phoenix 等 eval 工具	不做完整评测平台	交互调试 + RAG 快照
AgentPrism 等 UI 库	不只提供可视化组件	内置 Runner + 证据 + Diff 闭环

五、开源治理与复现方式

- 许可证：Apache-2.0
- 文档：README、用户手册、架构、对比说明、路线图、演示脚本、依赖说明、SECURITY
- 协作：CONTRIBUTING、Issue / PR 模板、CI (typecheck / test / build)
- 依赖合规：见 docs/DEPENDENCY-NOTICE.md

```
pnpm install
pnpm dev
```

无 API Key 时：左侧加载离线 Demo 或 OTLP 样例即可完成核心演示。

六、演示路径（约 2–3 分钟）

1. 预设 → 填 Key（录屏打码）→ 可选切换白天/夜间主题
2. 运行 → 进度、时间线增长、流式答案、Tokens/估成本
3. 点 retriever 看证据 → 「RAG 快照」或失败诊断 / Run Diff
4. 回放 → 导出 JSONL / MD

分镜详见 docs/DEMO-SCRIPT.md。

七、路线图与评审对照

阶段	内容
赛期已完成	四视图、Live Run、失败诊断、Diff、OTLP 薄导入、瀑布与关键路径、报告导出、主题、流式 usage、RAG 质检快照、单测与 CI
赛后规划	万级 span、完整 OTLP 桥、框架适配器、数据集与可选评测

评审维度	本项目对应
技术创新 30%	交互观测 + RAG 证据/质检 + Live Run + 失败诊断
场景落地 30%	可跑 Demo、Diff、导入导出、离线样例、演示脚本
开源治理 20%	Apache-2.0、文档体系、SECURITY、模板与 CI
长期发展 20%	赛期/赛后路线清晰，协议可扩展

附录：导出 PDF 建议配图

非强制；建议 2~4 张，Key 务必打码。

1. 工作台总览（Live Run + 中栏多视图）
2. 时间线或瀑布图（含关键路径 / token 线索）
3. RAG 证据面板（或质检快照内容）

4. Run Diff 或失败诊断

声明： 本作品为原创开源项目；第三方依赖按其许可证使用。自主选题，未绑定企业赛题。